Comparing KISS vs LangGraph Deep Research: An Empirical Exploration

I use GEPA through DSPy for prompt optimization. When I saw KISS had its own GEPA implementation, I wanted to understand how it works differently—and how a single-agent framework compares to LangGraph's multi-stage pipeline on real tasks. This is what the numbers showed.

AI Agents | Prompt Evolution | Benchmark Notes | Open Source

> **Quick take:** KISS was 3.1× faster and 2.8× cheaper than LangGraph's multi-stage pipeline, with a 1-point average quality gap favoring LangGraph (6.3 vs 7.3). On the widest question (7 tickers, macro risks), KISS actually scored *higher*. Prompt evolution via `prompt_refiner_agent` doubled token usage across runs without consistently lifting quality—the refiner's heuristic was "gather more data," but more data didn't equal better output.

Why This Exploration

I've been using **GEPA (Genetic-Pareto prompt evolution)** through DSPy for a while—it's a remarkably effective technique for optimizing compound AI system prompts via natural language reflection instead of reinforcement learning (arXiv:2507.19457).

When I came across the KISS Agent Framework by Koushik Sen at UC Berkeley, two things caught my attention:

1. **KISS has a native GEPA implementation**—not through DSPy, but integrated directly into the framework alongside a lighter-weight `prompt_refiner_agent` that uses trajectory-aware reflection to evolve system prompts between runs.
2. **KISS takes a single-agent approach**—one ReAct agent with all tools, no sub-agent decomposition—which is a fundamentally different architecture from what LangGraph's Open Deep Research does.

I wanted to understand both dimensions empirically: how does single-agent vs. multi-agent compare on real tasks, and what happens when you layer KISS's prompt evolution on top? Rather than argue for one approach, I set up controlled experiments and collected the data.

GEPA: Two Implementations, Different Integration Points

For context on why I found this interesting—GEPA's core algorithm is straightforward:

1. Maintain a pool of prompt candidates
2. Evaluate on dev/val splits with instance-level tracking
3. Reflect on trajectories to propose mutations
4. Gate mutations (only accept if they don't regress on the minibatch)
5. Update a Pareto frontier balancing quality, cost, and latency

In DSPy, GEPA sits as an external optimizer—you define your pipeline, provide training examples with a metric, and GEPA evolves the prompts offline. The pipeline itself is unchanged at inference time.

In KISS, the integration is different. KISS provides two mechanisms:

- `GEPA` class (`kiss.agents.gepa`)—the full optimization loop with Pareto frontiers, mutation gating, and population management. Requires training examples and a scoring function, similar to DSPy's usage.
- `prompt_refiner_agent` (`kiss.agents.kiss`)—a lighter mechanism that takes the original prompt, the previous prompt, and the agent's execution trajectory, then produces a refined prompt via a single non-agentic LLM call. No training set required.

The `prompt_refiner_agent` is what you'd use between runs in a production setting where you don't have labeled training data—it learns from the agent's own execution trace. This is what I tested in the evolution benchmark below.

Experiment 1: KISS vs LangGraph on Real Finance Tasks

Setup

Both frameworks ran on **3 finance research questions** requiring real-time data retrieval and synthesis:

#	Question
Q1	Compare NVIDIA vs AMD as AI chip investments—prices, P/E, revenue growth, market positioning
Q2	Analyze Tesla stock fundamentals + search for robotaxi news impact on valuation
Q3	Risk assessment of all Magnificent 7 stocks—P/E, YTD returns, macro risks, overvaluation ranking

Controlled variables:

- Same model: Claude Opus 4.6 (Databricks-hosted endpoint)
- Same tool functions: `get_stock_info` (yfinance), `get_stock_history` (yfinance), `web_search` (DuckDuckGo API)
- Same quality judge: LLM-scored on data grounding, analysis depth, recency, and actionability (1–10 scale)
- Single run per question per framework (n=1 per cell)

Architecture Comparison

KISS uses `KISSAgent.run()`—a single ReAct agent that receives all tools as plain Python callables:

```
Question + Tools
       |
       v
[KISSAgent.run()] ---> calls tools as needed ---> Report
  (ReAct loop, up to 15 rounds)
```

LangGraph uses a compiled `StateGraph` reproducing the Open Deep Research 4-stage pipeline:

```
Question
  |
  v
[Research Brief] -----> 3 sub-topics
  |
  v
[Researcher 1]  [Researcher 2]  [Researcher 3]
 (tools x6)      (tools x6)      (tools x6)
  |               |               |
  v               v               v
[Compress Research]
  |
  v
[Final Report]
```

Tool functions are identical. KISS receives them as `[get_stock_info, get_stock_history, web_search]`. LangGraph wraps them via `@langchain_tool` decorators calling the same underlying functions.

Results: Single Run Comparison

3.1x	4.1x	2.8x	+1.0
KISS FASTER	FEWER TOKENS	LOWER COST	LG QUALITY EDGE

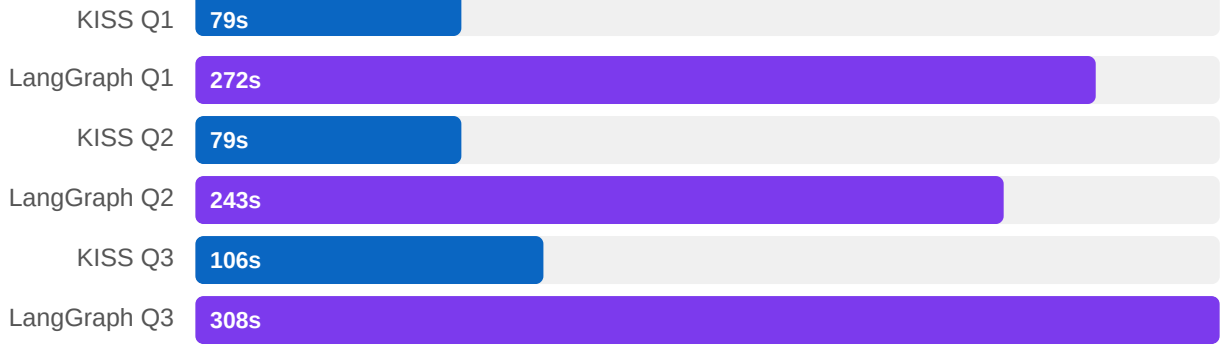
Metric	KISS	LangGraph ODR	Delta
Avg. Latency	88 s	274 s	3.1x
Avg. Tokens	20,793	85,449	4.1x
Avg. Cost	$0.73	$2.04	2.8x
Avg. Tool Calls	15.0	55.7	3.7x
Avg. Quality (1-10)	6.3	7.3	LG +1.0
Avg. Output Length	6,533 chars	14,019 chars	2.1x
Questions Passed	3/3	3/3	Equal

Per-Question Breakdown

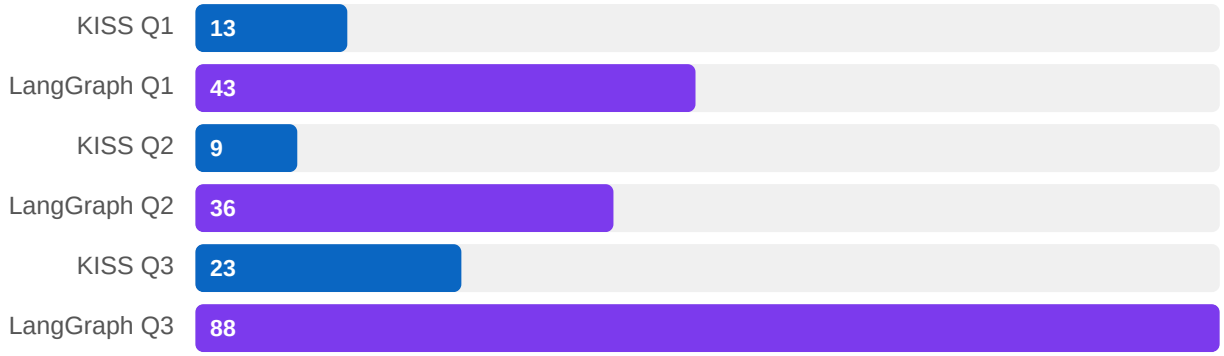
	Q1: NVDA vs AMD		Q2: Tesla		Q3: Mag 7	
Metric	**KISS**	**LG**	**KISS**	**LG**	**KISS**	**LG**
Time	79s	272s	79s	243s	106s	308s
Tokens	18.2K	78.4K	14.7K	53.6K	29.5K	124.3K
Cost	$0.64	$1.94	$0.52	$1.48	$1.03	$2.70
Tool Calls	13	43	9	36	23	88
Quality	7	8	5	8	7	6

> **Q2 gap (KISS 5, LangGraph 8)—the largest single-question difference:** KISS made only 9 tool calls on the Tesla question—it retrieved fundamentals but under-invested in the robotaxi news angle, producing a report the judge scored low on recency and actionability. LangGraph's 36 tool calls included dedicated `web_search` passes across all three researchers, surfacing recent regulatory and Waymo coverage. This points to a structural difference: KISS's ReAct loop, given a "fetch fundamentals first" signal in the question, deprioritized search; LangGraph's parallel researchers independently ran news passes regardless. A task-specific system prompt—or an evolved one—could close this gap.

Latency per Question



Tool Calls per Question



> **Observation:** Q3 (Magnificent 7, 7 tickers) is notable. LangGraph's three researchers each independently pulled data for all 7 tickers—88 tool calls, 124K tokens. KISS used 23 tool calls and scored 7/10 vs LangGraph's 6/10 on this specific question. On Q1 and Q2, LangGraph scored higher. Sample size is n=1 per cell.

Experiment 2: KISS Prompt Evolution Across Runs

This is the experiment I was most curious about. KISS provides a `prompt_refiner_agent` that takes the execution trajectory from a completed run and produces a refined system prompt. The mechanism is straightforward—a non-agentic LLM call that analyzes what the agent did, what worked, what didn't, and proposes prompt modifications.

I wanted to see: **does quality improve when you feed the trajectory back?**

Protocol

1. **Run 1 (Baseline):** Static system prompt. Record output, quality score, and full trajectory.
2. **Evolve:** Feed trajectory into `prompt_refiner_agent(original_prompt, previous_prompt, trajectory)`. Get refined prompt.
3. **Run 2 (Evolution #1):** Use refined prompt. Record everything. Evolve again.
4. **Run 3 (Evolution #2):** Use double-evolved prompt. Record everything.

Same 3 questions. Same tools. Same model. The only variable changing between runs is the system prompt.

Results: Evolution Across 3 Runs

Metric	Baseline	Evolution #1	Evolution #2
Avg. Quality (1-10)	6.7	6.7	6.3
Avg. Latency	97.9s	106.2s	117.5s
Avg. Tokens	15,634	31,559	32,114
Avg. Cost	$0.55	$1.10	$1.12
Avg. Tool Calls	13.3	20.7	21.0
Avg. Output Length	9,611 chars	10,486 chars	4,664 chars
Questions Passed	3/3	3/3	3/3

Quality Per Question Across Runs

Question	Baseline	Evo #1	Evo #2	Trend
Q1: NVDA vs AMD	7	7	5	-2
Q2: Tesla	6	6	7	+1
Q3: Mag 7 Risk	7	7	7	0

What the Evolved Prompts Look Like

The `prompt_refiner_agent` made the system prompt progressively more detailed across iterations. The baseline prompt was 6 lines. After two evolution passes:

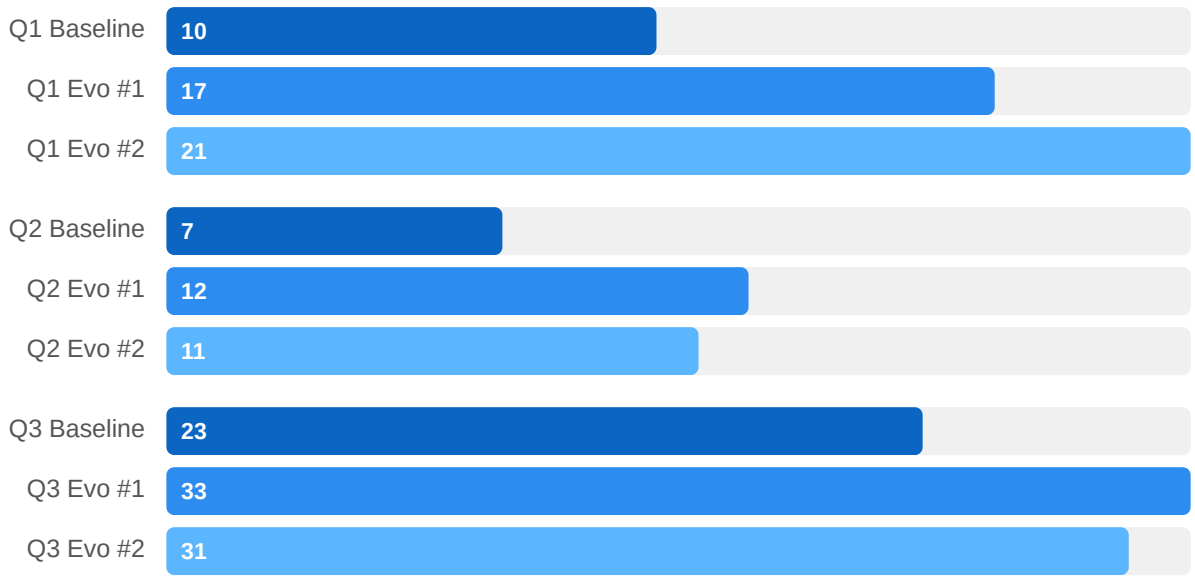
- Added: "Call `get_stock_info` once per ticker separately"
- Added: "Make all tool calls early—do not interleave lengthy analysis between tool calls"
- Added: "If a tool call fails, retry with adjusted parameters or use web_search as fallback"
- Added: "Cite the source (e.g., 'per get_stock_info') and note the date/time context"
- Added explicit section structure requirements (Executive Summary, Individual Analysis, Comparative Analysis, etc.)

The prompt grew from ~400 characters to ~1,800 characters.

Why did the refiner converge on "gather more data" as its improvement heuristic? The answer is visible in the baseline trajectory: the agent would retrieve two or three tickers' data, begin its analysis mid-loop, then realize it was missing the remaining tickers and either estimate or skip them. The refiner read this gap and added explicit per-ticker instructions with an early data-gathering phase. The diagnosis was accurate—data coverage was a real baseline weakness.

But the refiner had no mechanism to notice that expanding the tool call budget was eating into the output depth budget. That's exactly what GEPA's mutation gating would catch: if a candidate prompt increases tool calls without improving the quality score on a held-out validation set, the mutation is rejected. Without that gate, the refiner optimizes the proxy (data coverage) rather than the objective (output quality).

Tool Calls Per Question Across Runs



> **Observation:** The evolved prompts consistently increased tool call volume (13.3 → 20.7 → 21.0 avg). The refiner identified "fetch more data" as the primary improvement vector. Tokens roughly doubled (15.6K → 32.1K). Quality did not track with thoroughness in this run—Q1 regressed, Q2 improved, Q3 was flat.

Observations (Not Conclusions)

The data raises several questions I find interesting but don't have enough runs to answer definitively:

On single-agent vs. multi-agent

- KISS was consistently faster and cheaper by significant margins (3.1x, 2.8x). This held across all 3 questions.
- LangGraph scored higher quality on 2 of 3 questions (Q1, Q2), lower on 1 (Q3). The average favored LangGraph by 1.0 point.
- The Q3 result (KISS 7, LG 6) is the most interesting data point—on the widest-scope question (7 tickers + macro risks), the multi-agent approach's 88 tool calls and 124K tokens correlated with *lower* quality, not higher. Whether this is signal or noise at n=1 is an open question.

- LangGraph's researchers don't share context. For Q3, `get_stock_info("NVDA")` was called 4 times across different researchers. KISS called it once.

On prompt evolution

- The `prompt_refiner_agent` produced substantively different prompts each iteration—not just paraphrases. It identified real patterns from the trajectory (tool call sequencing, data coverage gaps, output structure).
- The evolved prompts pushed the agent toward more tool calls and more data gathering, but this didn't consistently translate to higher quality scores.
- Q1 regressed from 7 to 5 on the third run. The quality judge noted the report was "essentially a summary paragraph rather than a deep comparative analysis"—the agent spent its token budget on 21 tool calls and didn't have room for a full report.
- Q2 improved from 6 to 7 on the third run. The judge noted it had "a specific entry range ($340-350) and risk factors"—more actionable than the baseline.
- This is the `prompt_refiner_agent` (single-pass reflection), not the full GEPA optimizer. GEPA with Pareto frontiers, mutation gating, and a validation set would behave differently—specifically, it would catch regressions like Q1's and gate them out.

On the GEPA/DSPy comparison

- DSPy's GEPA implementation operates offline with a training set and metric function. KISS's `prompt_refiner_agent` operates online from a single trajectory—no training data required.
- The trade-off is clear: DSPy-GEPA can optimize against a known objective; KISS's refiner can adapt without one. For production settings without labeled data, the trajectory-based approach is more immediately applicable.
- I haven't tested KISS's full `GEPA` class (the one with Pareto frontiers) on this benchmark yet. That would be a more apples-to-apples comparison with DSPy's implementation and would likely show different evolution dynamics.

If You Had to Pick

With n=1 caveats firmly in place, the data points toward a practical heuristic. **For latency- or cost-constrained workflows where "good enough" quality is acceptable, KISS's single-agent approach is hard to beat**—88 seconds and $0.73 vs 274 seconds and $2.04 for comparable output. **For breadth-first research where sub-questions are genuinely independent**, LangGraph's parallel pipeline earns its overhead; the dedicated researcher-per-subtopic model helped on Q1 and Q2. The Q3 result suggests the flip side: when sub-questions *aren't* independent (all 7 tickers share the same tools and context), the parallelism backfires into redundant work and context fragmentation.

On prompt evolution: the `prompt_refiner_agent` is a useful diagnostic—it accurately identifies what your agent is under-doing in the current run. But without a quality gate it optimizes for the proxy rather than the objective. Treat it as a first-pass signal about where your system prompt has gaps, not as a self-contained optimizer. For that, you want the full GEPA loop with a validation set and mutation gating.

Open Questions for Further Exploration

- With more runs (n=5+), does the prompt evolution show a clearer quality trend? The current n=1 per cell is noisy.
- Does KISS's full GEPA optimizer (with Pareto frontiers and mutation gating) produce different evolution dynamics than the single-pass refiner?
- The quality judge is the same model doing the work—how much does this bias the scores?
- Would a hybrid approach (KISS-style single agent with LangGraph-style explicit data gathering phases) capture the efficiency of one and the coverage of the other?
- How do these results change with a smaller/cheaper model (Haiku, Sonnet) where the cost differential matters more?
- Can the prompt evolution budget be constrained (e.g., "improve quality but keep tool calls under 15") to prevent the thoroughness-at-the-expense-of-output regression seen in Q1?

Reproduce These Results

Both benchmarks are open source. The code uses actual framework imports (`KISSAgent.run()`, `StateGraph.compile()`), not API simulations.

```
# Clone and install (requires Python 3.13+ for KISS)
git clone https://github.com/tamtam0/kiss-vs-langgraph.git && cd kiss-vs-langgraph
pip install kiss-ai anthropic yfinance langgraph langchain-anthropic duckduckgo-se

# Set your API key
export ANTHROPIC_API_KEY="your-key"

# Experiment 1: KISS vs LangGraph (single run)
python3 benchmarks/bench_real.py

# Experiment 2: KISS prompt evolution (3 runs per question)
python3 benchmarks/bench_evolution.py

# Results:
# benchmarks/benchmark_real_results.json
# benchmarks/benchmark_evolution_results.json
```

Both scripts support Databricks-hosted endpoints via `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN` environment variables. Edit `BENCHMARK_QUESTIONS` in either file to test your own scenarios.

> **Methodology note:** All results from single benchmark runs, March 2026. Quality scores are LLM-judged (same Claude Opus 4.6 model, same rubric for both frameworks). Tool calls use live Yahoo Finance data and DuckDuckGo web search. n=1 per cell for the framework comparison, n=3 sequential runs per question for the evolution experiment. Results will vary with different models, market conditions, and question complexity. Full code and raw data on GitHub.

Benchmark code & data: github.com/tamtam0/kiss-vs-langgraph
KISS Framework: github.com/ksenxx/kiss_ai (Apache 2.0)
LangGraph ODR: github.com/langchain-ai/open_deep_research
GEPA Paper: arXiv:2507.19457 — Reflective Prompt Evolution Can Outperform Reinforcement Learning
DSPy: github.com/stanfordnlp/dspy